



**LANDMARK METROPOLITAN
UNIVERSITY INSTITUTE**
THE PRIDE OF AFRICA

INTRODUCTION TO PYTHON

Python Programming with Project

Precious Oben, 2025/2026

LANDMARK DIGITAL LESSONS

(+237) 672 339 570 / 694 990 622 | www.landmark.cm



WHAT IS PYTHON?



Python is a widely used High Level Programming Language that has been around since the late 1980s. The language places strong emphasis on code readability and simplicity, making it possible for programmers to develop applications rapidly.

It supports multiple programming paradigms, including:

- **Procedural programming** (step-by-step instructions)
- **Object-oriented programming (OOP)** (working with objects)
- **Functional programming** (using functions as building blocks)

Python is widely used across various fields, such as **web development, data science, machine learning, automation, game development, and cloud computing**. Its extensive standard library and a vibrant ecosystem of third-party packages make it versatile and powerful for almost any programming task.

With Python, developers can write fewer lines of code to achieve more, thanks to its concise and human-readable syntax. This combination of simplicity and functionality has led Python to become one of the most in-demand and widely adopted languages worldwide.

WHY PYTHON?



- **Readable Syntax:** Python's syntax is simple and in plain English, making it easy to learn, especially for beginners.
- **Minimal Code:** Tasks that require many lines of code in other programming languages are comparatively smaller in Python.
- **Versatility:** Python is used in various fields: web development, data science, machine learning, artificial intelligence, automation, scripting, cybersecurity and more.
- **Extensive Libraries and Frameworks:** Python has an **impressive collection of libraries** and frameworks, such as:
 - **Data Science & Machine Learning:** NumPy, Pandas, TensorFlow, PyTorch
 - **Web Development:** Django, Flask, FastAPI
 - **Automation:** Selenium, OpenCV, Paramiko

These libraries save time and effort, letting developers focus on solving problems rather than reinventing solutions.

- **Community Support:** Python has a large and active community, which means abundant resources, tutorials, forums, and open-source contributions. This makes it easy to find help, share ideas, and stay updated on the latest developments.
- **Cross-Platform and Open-Source:** Python is free and open-source, with support across multiple platforms (Windows, macOS, Linux). It ensures developers can use Python on any operating system and deploy applications anywhere.

APPLICATIONS OF PYTHON?



- **Web Development** – Django, Flask, FastAPI
- **Data Science and Analysis** – Pandas, Numpy
- **Machine Learning & AI** – TensorFlow, PyTorch
- **Automation & Scripting** – Selenium, Paramiko
- **Game Development** – Pygame
- **Cybersecurity** – Ethical hacking, Nmap scripts
- **Internet of Things (IoT)** – MicroPython
- **Desktop Applications** – Tkinter, PyQt
- **Cloud Computing** – AWS Lambda, Azure Functions
- **Data Visualization** – Matplotlib, Seaborn

GETTING STARTED



- Install Python 3 interpreter from <https://www.python.org/downloads/> after selecting a package that works with the OS and Processor of your PC.
- To ensure successful installation, on the command line, type the following.

```
python --version
```

- Write your first line of Python Code

```
print("Hello World!")
```

PYTHON SYNTAX



As explained before, Python is one of the easiest programming languages due to the fact that it can be read easily like English.

However, there is a set of rules which defines how a Python program should be written and would be interpreted. This is called the **Syntax**.

1. Indentation:

Indentation (usually 4 spaces) is mandatory to define the scope of loops, functions, and conditionals. Incorrect indentation will result in an error.

```
for i in range(3):    <----- Correct
    print(i)
```

```
for l in range(3):    <----- Incorrect
print(i)
```

PYTHON SYNTAX



2. Data types and Variables:

There is no need to declare variable types explicitly. Python infers the type dynamically.

Example:

```
name = "Alice" # String  
age = 25 # Integer  
height = 5.9 # Float
```

3. Comments

Use # for single-line comments and triple quotes (''' or """) for multi – line comments.

4. Printing Output

Use the print() function to display output.

5. No Semicolons or Curly Braces

Python uses indentation to define code blocks instead of curly braces {}
No need for semicolons at the end of each line.

VARIABLES



Variables are names given to data we need to store and manipulate in programs. For instance, suppose your program needs to store the age of a user. To do that we can use a variable called “userAge” to store this data.

```
userAge = 0
```

After defining this variable, your program will allocate a certain area of your PC’s storage space to store this data. You can then access and modify this data by referring to its name.

Every time you declare a new variable, you need to give it an initial value which can always be changed later. Multiple variables can also be defined at one go.

```
userAge, username = 30, 'Albert'
```

This is equivalent to

```
userAge = 30
```

```
userName = 'Albert'
```

VARIABLES



NAMING CONVENTIONS

A variable name in Python can only contain letters (a – z, A – Z), numbers or underscores (_). However, the first character cannot be a number.

In addition, there are some reserved words that cannot be used as variable names because they already have preassigned meanings in Python. These are words like input, print, if, while etc.

Finally, variable names are case sensitive: Username is not the same as username.

There are two conventions when naming a variable in Python. We can either use the **Camel case notation** or use **underscores**. Camel case is the practice of writing compound words with mixed casing e.g totalNumberOfCars. Using underscores requires the words to be separated by underscores.

DATA TYPES



We'll explore the basic data types in Python, specifically the integer, float, string and boolean. Further in the course more advanced types like the list, tuple and dictionary will be addressed.

1. Integers

Integers are numbers with no decimal parts e.g -5, 0, 6, 1 etc

To declare an integer in Python, simply initialize your variable name.

Example:

```
numberOfDays = 3
```

2. Float

Float refers to numbers that have decimal parts, such as 1.234, -0.023, 12.01. To declare a float in Python, simply initialize the variable name.

3. String

String refers to text. To declare a string, you can either use single or double quotes when initializing variables
`userName = 'Peter'`

DATA TYPES



Built-In String Functions

Python has some built-in methods to manipulate strings. An example is one used to capitalize string.

`"Peter".upper()` should return `"PETER"`.

Formatting Strings using the % Operator

Strings can be formatted using the % operator. It gives greater control over how you want your string to be displayed. Its syntax is:

`"string to be formatted" %(values or variables to be inserted into string, separated by commas)`

Example

```
brand = "Apple"
exchangeRate = 1.235235245
message = "The price of this %s laptop is %d USD and the exchange rate is %4.2f USD to 1 EUR" % (brand, 1299,
exchangeRate)
print(message)
```

DATA TYPES



4. Boolean

Booleans represent one of two values, True or False.

When two values are compared in python, the response is a boolean value. For example:

```
print(5>9)
```

When a condition is run in an IF Statement, Python returns True or False

Example

```
a = 200
```

```
b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
else:
```

```
    print("b is not greater than a")
```

OPERATORS IN PYTHON

Operators are used to perform operations on variables and values.
There are different groups of operators in Python:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators

1. Arithmetic Operators: **+, -, *, /, %, **, //**
2. Assignment Operators: **=, +=, -=, %=, etc.**
3. Comparison operators: **==, !=, >, <, >=, <=**
4. Logical Operators: **and, or, not**
5. Identity Operators: **is, is not**

CONTROL STRUCTURES



Control structures allow you to dictate the flow of your program based on certain conditions or repetitions. They help manage **decision-making** and **loops**, ensuring that specific code blocks are executed only when necessary. Python supports several control structures, such as conditionals, loops, and exception handling.

1. CONDITIONAL STATEMENTS

Conditional statements allow programs to make decisions and execute code based on certain conditions. In Python, we use **if**, **elif** and **else** to handle these scenarios.

Example:

```
x = 10
if x > 0:
    print("Positive")
elif x < 0:
    print("Negative")
else:
    print("Zero")
```

CONTROL STRUCTURES



Python supports the logical conditions from mathematics:

- Equals: $a == b$
- Not Equals: $a != b$
- Less than: $a < b$
- Less than or equal to: $a \leq b$
- Greater than: $a > b$
- Greater than or equal to: $a \geq b$

Example

```
a = 30
b = 30
if a == b:
    print("b equals a")
else:
    print("b does not equal a")
```

NOTE : Take note of indentation

EXERCISES



1. Write a program that:

-Asks the user to enter a number between 1 and 9.

-If the entered number is even, it prints:

“You entered an even number”

-If the number is odd, it prints:

“You entered an odd number”

-If the number is out of range, print:

“Invalid input! Enter a number between 1 and 9”

2. Write a program that returns the number of letters in a user’s name. (Hint: use len())

3. Write a program that asks for a user’s first name, then second name and returns both.